Part 1. Software Security

Alejandro Panigada
2003597

1. Context

ScottishGlen is a small company within the energy sector. The only IT manager oversees the team of 6 developers who are responsible for the company's suite of applications, as well as some technical staff who attend to infrastructure. No one is specifically responsible for security.
Employees have been receiving messages from a hacktivist group, who threaten to target the company following a recent blog post by the CEO. The nature of the attack is unknown.
The CEO is concerned and has asked the IT manager to improve the security posture to better protect the company. The focus is on making the company resilient to attack, from both a technical (systems) and human (employee) perspective.

This document describes from the point of view of the IT manager what is the most crucial system which could be used during an attack and how to prevent a potential incident to occur.

Kerberos implementation on Microsoft windows was the system selected to conduct specific analysis. This is due to several attack vectors being present and a large number of vulnerabilities being discovered compared to other software in the company.

Microsoft implementation is also the one assumed to be used in the company because of the other software being run in their environment being based on other Microsoft technologies such as ASP.NET and Windows GUI frontend.

From DoS attacks, up to full privilege escalation exploits, as well as sensitive information disclosure, Microsoft implementation of Kerberos is the version which has been observed to have a higher number of CVEs compared to other Kerberos versions such as MIT Kerberos (20 CVEs identified for the Microsoft implementation across a sample of 50).

Among the many attacks available, the most common ones were found to be privilege escalation and information disclosure.

Several of the privilege escalation CVEs were found to have the same cause of other CVEs marking the presence of persistent insecure patterns during coding.

The report centres around the privilege escalation vulnerability of the Microsoft implementation of Kerberos.

2. Recommendation

Kerberos implementation of Microsoft is closed source, so the team at ScottishGlen cannot possibly modify it and patch it freely as pleased.

Also, the number of people in the team, including the IT manager, is small, so no excessive amount of work can be performed on the software even if code was available and especially not when the solution is already at the maintenance stage and the problem should have been addressed at the design stage of the development life cycle.

Many are the attacks possible to the Kerberos authentication system, but the ones covered in this report focus on the cryptographic components and flawed application logic in Windows and Windows server before November 11, 2025 (Microsoft Corporation,2025).

The main recommendation for addressing in general, not only for Kerberos, is to ensure that all cryptographic modules used by the final application/system should be compliant to FIPS 140-2, NIST standard specifying approved cryptographic algorithms and modules, or an equivalent standard.

This should be checked for every possible module, new or old, and for every software library.

This might be time consuming for an already deployed system and difficult to perform for tightly integrated software such as Kerberos and Windows Active Directory.

However, by having a high standard defined for encryption modules to be used in the final product many attacks can be prevented.

Also, in some software, such as Kerberos, if replacement is not possible, at the very least the configuration can be changed to avoid usage of insecure protocols and ciphers.

The CVE later to be described heavily relied on excessive trust from the developers and allowance of insecure cryptographic functions which allows after an initial intrusion, to further escalate both vertically and horizontally in the network.

In the case of an attack to the ScottishGlen company, risks are not only about exploiting easier to reach machines or devices, but also of attackers escalating privileges once they are inside further compromising the insides of the network.

Enforcing a policy requiring all cryptographic modules to adhere to a well-defined cryptographic standard can mitigate such vulnerabilities before damage is brought to the company's systems.

3. Implementation

One example of attack where the usage of the FIPS 140-2 or an equivalent standard as baseline for cryptographic module evaluation can prevent damage is CVE-2025-60704 (MITRE,2025; Microsoft Corporation,2025).

The vulnerability allows privilege escalation by a combination of a cryptographic downgrade, missing server-side validation, and a client-side integrity check bypass.

The exploit is based around the delegation system used by Kerberos on Windows.

Delegation is the mechanism of delegating the access of a backend resource on behalf of the user by the frontend. (Neuman et al., 2005)

This is done to maintain authorization context without having to copy credentials or implement application-level impersonation.

There are two types of delegation: Constrained and Unconstrained.

The unconstrained delegation just takes the TGT and forward it. Once trusted, the service can impersonate the user to any service inside the domain but that can be dangerous in case of compromising of the network.

Constrained delegation instead of relying on the forwarding of the TGT, is based on making the frontend authorized to impersonate only to specific services using a service ticket.

On Windows it is also present the Service-for-user(S4U) protocol extension where two mechanisms are introduced: Protocol transition (S4USelf) and constrained delegation(S4UProxy) (Microsoft Corporation,2022).

S4USelf is used when the user does not authenticate to an application using Kerberos, but the application requires a Kerberos ticket to represent the user's identity.

S4UProxy serves the purpose of getting a service ticket which is forwardable by providing previous authentication evidence such as an old service ticket or by presenting a service ticket retrieved from S4USelf.

The attack is carried out without possession of any credential.

Only a single field, the username, is enough to carry out this attack.

The attack combines 3 key vulnerabilities:

Encryption Algorithm Downgrade

The attacker, positioned between the client and the KDC via a Machine-in-the-Middle position, intercepts the AS-REQ request and downgrades the TGT encryption type to RC4-HMAC. Because the default AS-REQ request is plaintext with no signing or sealing, downgrading requires no special privileges. Once the TGT session key is RC4-based, the checksum protecting the PA-S4U-X509-USER structure degrades to RSA-MD4, which results in an unkeyed checksum where integrity is computed using only MD4 algorithm with no session key involved. Because the checksum is unkeyed, the attacker can inject a crafted data structure, called PA-DATA type 130, into the S4U2Self TGS-REQ request carrying an attacker-chosen identity, compute a valid MD4 checksum over it without any secret key, and have the KDC accept the forged identity assertion as valid.

Missing Server-Side Validation — CWE-325 (MITRE, 2024)

When processing the S4U2Self request, the KDC does not perform any validation of the negotiated encryption type and no restriction on which PA-DATA structures are acceptable are present. While the legitimate client will never send PA-DATA type 130 when RC4 is the session key type, the server side does not prevent an attacker from injecting it. Because there is no cryptographic binding

between the session key type and the PA-DATA structure used to assert identity, the KDC selects RSA-MD4 as the checksum algorithm, verifies the attacker-supplied checksum, and issues a valid S4U2Self service ticket for the attacker-chosen privileged identity. This is categorized as a CWE-325 violation in which a cryptographic step allows a forged identity assertion with nothing preventing it from happening.

Client-Side Integrity Check Bypass

The ticket issued however, fails to install on the client because of the existence of an integrity mechanism on the client side. When RC4 is used, the KDC embeds both the request and reply to checksum values inside the encrypted portion of the TGS-REP reply, and the client is expected to verify both before accepting the ticket. This check is only triggered if PA-DATA type 130 is present in the reply. By removing PA-DATA type 130 from the encrypted section of the TGS-REP reply entirely, the client follows a code branch which returns success without performing any checksum verification (Partush, 2026). To finalize the exploit, the outer cname field of the ticket must be restored to the original user's identity. Cname carries no integrity check, so it is possible to modify it. The result is a ticket that installs successfully into the original user's logon session while the PAC embedded within it represents the attacker-chosen privileged identity.

To summarize:

- Attacker intercepts AS-REQ via MitM → downgrades TGT encryption type to RC4 → KDC issues TGT with RC4 session key
- Attacker injects crafted PA-DATA type 130 into S4U2Self TGS-REQ → server performs no validation of encryption type against PA-DATA structure → KDC selects RSA-MD4 as checksum algorithm
- Attacker computes MD4 checksum over attacker-chosen identity → KDC accepts forged identity assertion → issues S4U2Self ticket for attacker-chosen user
- Attacker removes PA-DATA type 130 from TGS-REP encrypted section → client skips compensating integrity check entirely → ticket installs
- Attacker restores outer cname to original user to match logon session expectations → PAC inside ticket still represents attacker-chosen privileged identity
- S4U2Proxy presents manipulated ticket → KDC derives downstream ticket identity from forged PAC → issues legitimate service ticket as privileged user
- Full domain privilege escalation achieved

Enforcing FIPS 140-2 compliance would have meant having MD4 disabled entirely and without MD4 available for negotiation, the attacker cannot force a weak unkeyed checksum. Without a weak unkeyed checksum, identity fields cannot be freely modified — any manipulation would invalidate a keyed HMAC-based checksum which cannot be recomputed so the first attack vector would have been neutralized.

FIPS compliance alone does not fix flawed checks. However, not much can be done in general as Microsoft is the only entity which can fix the fault with a patch. However, the missing validation checks on server side and check bypass on client-side are logic flaws independent of algorithm

strength. Also, the attacker has no mechanism to produce a manipulated service ticket with a valid checksum. This makes the flawed checks in the code irrelevant.

This is an example on how the use of cryptographic compliance standard can prevent an attack which otherwise could lead to severe compromising of the network. Adhering to a single standard can sometimes even make potential vulnerabilities in code useless and unusable against the network.

While other practices such as penetration testing and compliance testing/security auditing can provide information on the overall status of the program/network, they do no enforce specifics measures but instead they are just recommended.

By having a predetermined enforced standard for cryptographic modules, the company has to strictly comply to any findings without leaving the freedom of a dangerous choice.

Also, FIPS 140-2 compliance enforcement does not only help identify vulnerabilities. It also has preventive capabilities as any module added or any modification is made to any component, before entering production it has to comply with the enforced rules.

References

Forshaw, J. (2022) RC4 is Still Considered Harmful. Google Project Zero. Available at: https://projectzero.google/2022/10/rc4-is-still-considered-harmful.html (Accessed: 18 February 2026).

Microsoft Corporation (2022) MS-SFU: Kerberos Protocol Extensions: Service for User and Constrained Delegation Protocol. Microsoft Open Specifications. Available at: https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-sfu/3bff5864-8135-400e-bdd9-33b552051d94 (Accessed: 18 February 2026).

Microsoft Security Response Center (2025) CVE-2025-60704: Windows Kerberos Elevation of Privilege Vulnerability. Available at: https://msrc.microsoft.com/update-guide/vulnerability/CVE-2025-60704 (Accessed: 18 February 2026).

MITRE (2024) CWE-325: Missing Cryptographic Step. Common Weakness Enumeration, Version 4.19.1. Available at: https://cwe.mitre.org/data/definitions/325.html (Accessed: 18 February 2026).

MITRE (2025) CVE-2025-60704: Common Vulnerabilities and Exposures Record. Available at: https://www.cve.org/CVERecord?id=CVE-2025-60704 (Accessed: 18 February 2026).

Neuman, C., Yu, T., Hartman, S. and Raeburn, K. (2005) RFC 4120: The Kerberos Network Authentication Service (V5). Internet Engineering Task Force. Available at: https://datatracker.ietf.org/doc/html/rfc4120 (Accessed: 18 February 2026).

NIST (2001) FIPS 140-2: Security Requirements for Cryptographic Modules. National Institute of Standards and Technology. Available at: https://csrc.nist.gov/publications/detail/fips/140/2/final (Accessed: 18 February 2026).

Partush, E. (2026) CVE-2025-60704: Validation Flaws in Windows Kerberos S4U: From Protocol Transition to Privilege Escalation. Silverfort. Available at: https://www.silverfort.com/resources/cve-2025-60704/ (Accessed: 18 February 2026).

Raeburn, K. (2005) RFC 3961: Encryption and Checksum Specifications for Kerberos 5. Internet Engineering Task Force. Available at: https://datatracker.ietf.org/doc/html/rfc3961 (Accessed: 18 February 2026).